

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure, Environment, Actuators, Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

The environment state variables are `self.width` and `self.height`, `self.walls`, `self.food_positions`, `self.opponents`, and `self.agent_pos`. `self.collision` is an event flag that the environment sets. `self.score` is the performance measure, and `self.steps` is a time counter used for termination, so neither is environment state.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Multi-agent. `self.opponents` holds other entities that move on their own each step and affect the agent's performance: a collision costs -50 and ends the game. The opponents are separate actors in the same environment, not just passive obstacles. One caveat: the `_main_` block runs with `num_opponents=0`, which is effectively single-agent, but the class default (`num_opponents=2`) is what defines the baseline.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything the agent has perceived from its sensors up to the current moment. A rational agent's choice of action can depend on this whole history.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what

the agent can and cannot see.

Partially Observable. The dictionary returns the agent's position, the opponents' positions, smells_food, hit_wall, collision, score and the count of remaining food. It does not reveal where the walls are, where the food is, or the grid size. smells_food only covers the agent's own cell. Because the agent can't see the full state, it must explore or remember.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance measure. Deducting points for hitting a wall changes the score, and the score is the performance measure.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

The performance measure should reflect what we actually want in the environment, not how we think the agent should behave internally. If it scored internal effort, the agent could be rewarded for looking busy or gaming the metric while the world ends up in a bad state. Scoring external outcomes, like hitting walls or collecting food, keeps the metric objective and independent of the agent's design.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes more partially observable. The traps are part of the true environment state, but the sensors don't reveal them, so the agent can't determine the complete state from its percepts. It has to explore, remember where it was penalized, or infer trap locations. If the baseline was already partially observable, it stays that way, with more hidden information.



Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos)` in `self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Rationality means choosing the action that maximizes expected performance given the percept sequence. `smells_toxin` adds a percept that is directly tied to a -15 penalty, so the agent can use it to update its internal model and mark that cell as dangerous. It can then avoid re-entering the trap and route around it, which raises expected utility.



Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic vacuum-world exploit: if the agent is rewarded for the amount of dirt cleaned up, a rational agent can clean a square, dump the dirt back out, and clean it again, repeatedly, maximizing the score without achieving a clean environment. The fix is to reward the state of the world, such as one point per clean square per time step, rather than an action that can be repeated.



Finalize and Lock Lab 01 Analytical Evaluation



FACULTY OF COMPUTING